Status	Finished
Started	Friday, 30 October 2026, 9:17 PM
Completed	Friday, 30 October 2026, 9:28 PM
Duration	10 mins 34 secs
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Given a tuple and a positive integer k, the task is to find the count of distinct pairs in the tuple whose sum is equal to **K**.

Examples:

> **Input:** t = (5, 6, 5, 7, 7, 8), K = 13
> **Output:** 2
> **Explanation:**
> Pairs with sum K(= 13) are {(5, 8), (6, 7), (6, 7)}.
> Therefore, distinct pairs with sum K(= 13) are { (5, 8), (6, 7) }.
> Therefore, the required output is 2.

For example:

Input	Result
1,2,1,2,5 3	1
1,2 0	0

Answer: (penalty regime: 0 %)

```
import sys

def main():
    # Read all inputs from standard input
    input_data = sys.stdin.read().strip().split('\n')
    if len(input_data) < 2:
        return

    # Parse the tuple elements and K
    t_str = input_data[0].strip()
    K = int(input_data[1].strip())

    # Convert comma-separated string to a tuple of integers
    t = tuple(map(int, t_str.split(',')))

    seen = set()
    distinct_pairs = set()

    for num in t:
        complement = K - num
        if complement in seen:
            # Sort the pair so that (a, b) and (b, a) are treated as the same pair
            pair = tuple(sorted((num, complement)))
            distinct_pairs.add(pair)
        seen.add(num)

    print(len(distinct_pairs))

if __name__ == '__main__':
    main()

```

	Input	Expected	Got	
✓	5,6,5,7,7,8 13	2	2	✓
✓	1,2,1,2,5 3	1	1	✓

	Input	Expected	Got	
✓	1,2 0	0	0	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

The **DNA sequence** is composed of a series of nucleotides abbreviated as `'A'`, `'C'`, `'G'`, and `'T'`.

- For example, `"ACGAATTCCG"` is a **DNA sequence**.

When studying **DNA**, it is useful to identify repeated sequences within the DNA.

Given a string `s` that represents a **DNA sequence**, return all the `10`**-letter-long** sequences (substrings) that occur more than once in a DNA molecule. You may return the answer in **any order**.

Example 1:

```
Input: s = "AAAAACCCCCAAAAACCCCCCAAAAAGGGTTT"
Output: ["AAAAACCCCC","CCCCCAAAAA"]
```

Example 2:

```
Input: s = "AAAAAAAAAAAAA"
Output: ["AAAAAAAAAA"]
```

For example:

Input	Result
AAAAACCCCCAAAAACCCCCCAAAAAGGGTTT	AAAAACCCCC CCCCCAAAAA

Answer: (penalty regime: 0 %)

```python
import sys

def findRepeatedDnaSequences(s):
    seen = set()
    res = set()

    # Iterate through the string using a window of length 10
    for i in range(len(s) - 9):
        sub = s[i:i + 10]
        if sub in seen:
            res.add(sub)
        else:
            seen.add(sub)

    return list(res)

def main():
    # Read the DNA sequence from standard input
    input_data = sys.stdin.read().strip()
    if not input_data:
        return

    s = input_data.split('\n')[0].strip()
    repeated_sequences = findRepeatedDnaSequences(s)

    # Print each sequence on a new line (or adjust based on online judge format)
    for seq in repeated_sequences:
        print(seq)

if __name__ == '__main__':
    main()
```

	Input	Expected	Got	
✓	AAAAACCCCCAAAAACCCCCCAAAAAGGGTTT	AAAAACCCCCCCCCCAAAAA	AAAAACCCCCCCCCCAAAAA	✓
✓	AAAAAAAAAAAAA	AAAAAAAAAA	AAAAAAAAAA	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

Write a program to eliminate the common elements in the given 2 arrays and print only the non-repeating elements and the total number of such non-repeating elements.

Input Format:

The first line contains space-separated values, denoting the size of the two arrays in integer format respectively.

The next two lines contain the space-separated integer arrays to be compared.

Sample Input:

5 4

1 2 8 6 5

2 6 8 10

Sample Output:

1 5 10

3

Sample Input:

5 5

1 2 3 4 5

1 2 3 4 5

Sample Output:

NO SUCH ELEMENTS

For example:

Input	Result
5 4 1 2 8 6 5 2 6 8 10	1 5 10 3
5 5 1 2 3 4 5 1 2 3 4 5	NO SUCH ELEMENTS

Answer: (penalty regime: 0 %)

```
import sys

def main():
    # Read all inputs from standard input
    input_data = sys.stdin.read().strip().split('\n')
    if len(input_data) < 3:
        return

    # Parse the two arrays
    arr1 = list(map(int, input_data[1].strip().split()))
    arr2 = list(map(int, input_data[2].strip().split()))

    set1 = set(arr1)
    set2 = set(arr2)

    non_repeating = []

    # Find elements in arr1 that are not in arr2
    for x in arr1:
        if x not in set2:
            non_repeating.append(x)

    # Find elements in arr2 that are not in arr1
    for x in arr2:
        if x not in set1:
            non_repeating.append(x)
```

```
27
28      # If no non-repeating elements exist, print the required message
29      if not non_repeating:
30          print("NO SUCH ELEMENTS")
31      else:
32          print(" ".join(map(str, non_repeating)))
33          print(len(non_repeating))
34
35  if __name__ == '__main__':
36      main()
37
```

	Input	Expected	Got	
✓	5 4 1 2 8 6 5 2 6 8 10	1 5 10 3	1 5 10 3	✓
✓	3 3 10 10 10 10 11 12	11 12 2	11 12 2	✓
✓	5 5 1 2 3 4 5 1 2 3 4 5	NO SUCH ELEMENTS	NO SUCH ELEMENTS	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

There is a malfunctioning keyboard where some letter keys do not work. All other keys on the keyboard work properly.

Given a string text of words separated by a single space (no leading or trailing spaces) and a string brokenLetters of all distinct letter keys that are broken, return the number of words in text you can fully type using this keyboard.

Example 1:

Input: text = "hello world", brokenLetters = "ad"

Output:

1

Explanation: We cannot type "world" because the 'd' key is broken.

For example:

Input	Result
hello world ad	1
Faculty Upskilling in Python Programming ak	2

Answer: (penalty regime: 0 %)

```
import sys

def main():
    # Read all inputs from standard input
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    text = input_data[0].strip()
    # Convert broken letters to lowercase for case-insensitive matching
    broken_letters = set(input_data[1].strip().lower()) if len(input_data) > 1 else set()

    words = text.split()
    typable_count = 0

    for word in words:
        # Check if any character in the word (converted to lowercase) is a broken letter
        if not any(char.lower() in broken_letters for char in word):
            typable_count += 1

    print(typable_count)

if __name__ == '__main__':
    main()

```

	Input	Expected	Got	
✓	hello world ad	1	1	✓
✓	Welcome to REC e	1	1	✓
✓	Faculty Upskilling in Python Programming ak	2	2	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

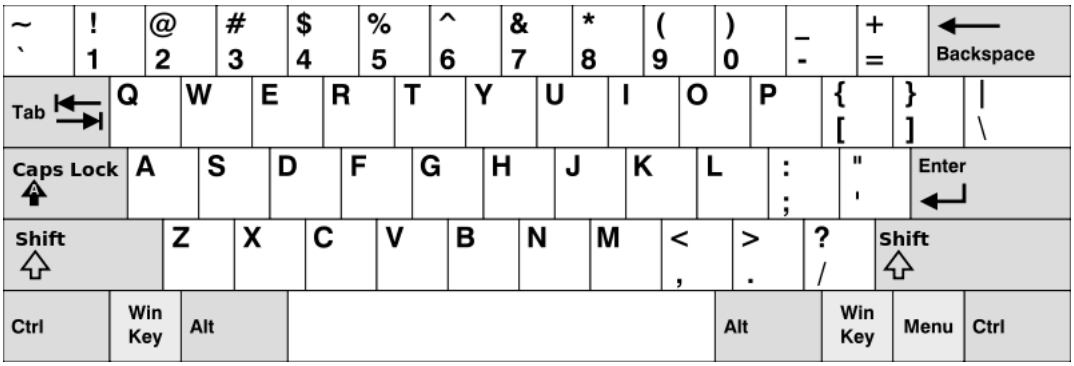
Correct

Mark 1.00 out of 1.00

Given an array of strings `words`, return *the words that can be typed using letters of the alphabet on only one row of American keyboard like the image below.*

In the **American keyboard**:

- the first row consists of the characters `"qwertyuiop"`,
- the second row consists of the characters `"asdfghjkl"`, and
- the third row consists of the characters `"zxcvbnm"`.



Example 1:

```
Input: words = ["Hello","Alaska","Dad","Peace"]
Output: ["Alaska","Dad"]
```

Example 2:

```
Input: words = ["omk"]
Output: []
```

Example 3:

```
Input: words = ["adsdf","sfd"]
Output: ["adsdf","sfd"]
```

For example:

Input	Result
4 Hello Alaska Dad Peace	Alaska Dad
2 adsfd afd	adsfd afd

Answer: (penalty regime: 0 %)

```
import sys

def findWords(words):
    row1 = set("qwertyuiop")
    row2 = set("asdfghjkl")
    row3 = set("zxcvbnm")

    result = []
    for word in words:
        w_lower = word.lower()
        if not w_lower:
            continue

        # Determine which row the first character belongs to
        first_char = w_lower[0]
```

```
        if first_char in row1:
            target_row = row1
        elif first_char in row2:
            target_row = row2
        else:
            target_row = row3

        # Check if all characters in the word belong to the same row
        if all(char in target_row for char in w_lower):
            result.append(word)

    return result

def main():
    # Read all inputs from standard input
    lines = sys.stdin.read().splitlines()
    if not lines:
        return

    # Handle optional leading count line if present
    start_idx = 0
    if lines[0].strip().isdigit():
        start_idx = 1

    words = [line.strip() for line in lines[start_idx:] if line.strip()]

    ans = findWords(words)

    # If no words match, print "No words" as required by the test case
    if not ans:
        print("No words")
    else:
        for w in ans:
            print(w)

if __name__ == '__main__':
    main()
```

	Input	Expected	Got	
✓	4 Hello Alaska Dad Peace	Alaska Dad	Alaska Dad	✓
✓	1 omk	No words	No words	✓
✓	2 adsfd afd	adsfd afd	adsfd afd	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

You are given an integer tuple `nums` containing distinct numbers. Your task is to perform a sequence of operations on this tuple until it becomes empty. The operations are defined as follows:

1. If the first element of the tuple has the smallest value in the entire tuple, remove it.
2. Otherwise, move the first element to the end of the tuple.

You need to return an integer denoting the number of operations required to make the tuple empty.

Constraints

- The input tuple `nums` contains distinct integers.
- The operations must be performed using tuples and sets to maintain immutability and efficiency.
- Your function should accept the tuple `nums` as input and return the total number of operations as an integer.

Example:

Input: nums = (3, 4, -1)
Output: 5

Explanation:
Operation 1: [3, 4, -1] -> First element is not the smallest, move to the end -> [4, -1, 3]
Operation 2: [4, -1, 3] -> First element is not the smallest, move to the end -> [-1, 3, 4]
Operation 3: [-1, 3, 4] -> First element is the smallest, remove it -> [3, 4]
Operation 4: [3, 4] -> First element is the smallest, remove it -> [4]
Operation 5: [4] -> First element is the smallest, remove it -> []
Total operations: 5

For example:

Test	Result
`print(count_operations((3, 4, -1)))`	5

Answer: (penalty regime: 0 %)

Reset answer

```
def count_operations(nums: tuple) -> int:
    # Sort the unique values to know the exact order in which elements will be removed
    sorted_vals = sorted(nums)

    # Map each value to its original index in the tuple
    val_to_idx = {val: i for i, val in enumerate(nums)}

    operations = 0
    n = len(nums)
    removed_set = set()
    current_pos = 0  # Tracks the current effective front position in the rotation

    for val in sorted_vals:
        target_idx = val_to_idx[val]

        # Count how many elements before target_idx have already been removed
        removed_before = sum(1 for r in removed_set if r < target_idx)
        effective_target = target_idx - removed_before

        # Calculate rotations needed to bring the target element to the front
        if effective_target >= current_pos:
            rotations = effective_target - current_pos
        else:
            remaining_count = n - len(removed_set)
            rotations = (remaining_count - current_pos) + effective_target

        # Add rotations + 1 step to remove the element
        operations += rotations + 1

        # Mark as removed and update current position for the remaining elements
        removed_set.add(target_idx)
        remaining_count = n - len(removed_set)
        current_pos = effective_target % remaining_count if remaining_count > 0 else 0

    return operations
```

```
36
```

	Test	Expected	Got	
✓	`print(count_operations((3, 4, -1)))`	5	5	✓
✓	`print(count_operations((1, 2, 3, 4, 5)))`	5	5	✓
✓	`print(count_operations((5, 4, 3, 2, 1)))`	15	15	✓
✓	`print(count_operations((42, )))`	1	1	✓
✓	`print(count_operations((-2, 3, -5, 4, 1)))`	11	11	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Coders here is a simple task for you, Given string str. Your task is to check whether it is a binary string or not by using python set.

Examples:

Input: str = "01010101010"

Output: Yes

Input: str = "REC101"

Output: No

For example:

Input	Result
01010101010	Yes
010101 10101	No

Answer: (penalty regime: 0 %)

```
import sys

def main():
    # Read the input string from standard input
    s = sys.stdin.read().strip()
    if not s:
        return

    # Convert the string to a set of its characters
    char_set = set(s)

    # Check if the character set contains only '0' and '1'
    if char_set.issubset({'0', '1'}):
        print("Yes")
    else:
        print("No")

if __name__ == '__main__':
    main()
```

	Input	Expected	Got	
✓	01010101010	Yes	Yes	✓
✓	REC123	No	No	✓
✓	010101 10101	No	No	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

Program to print all the distinct elements in an array. Distinct elements are nothing but the unique (non-duplicate) elements present in the given array.

Input Format:

First line take an Integer input from stdin which is array length n.

Second line take n Integers which is inputs of array.

Output Format:

Print the Distinct Elements in Array in single line which is space Separated

Example Input:

5

1 2 2 3 4

Output:

1 2 3 4

Example Input:

6

1 1 2 2 3 3

Output:

1 2 3

For example:

Input	Result
5 1 2 2 3 4	1 2 3 4

Answer: (penalty regime: 0 %)

```
import sys

def main():
    # Read all inputs from standard input, handling both space-separated and newline-separated values
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    # First value is the array length n (optional to use explicitly since we can parse the rest)
    n = int(input_data[0])
    arr = [int(x) for x in input_data[1:]]

    # Extract unique elements while preserving original order
    distinct_elements = []
    seen = set()

    for num in arr:
        if num not in seen:
            seen.add(num)
            distinct_elements.append(num)

    # Print the distinct elements space-separated on a single line
    print(" ".join(map(str, distinct_elements)))

if __name__ == '__main__':
    main()
```

	Input	Expected	Got	
✓	5 1 2 2 3 4	1 2 3 4	1 2 3 4	✓
✓	6 1 1 2 2 3 3	1 2 3	1 2 3	✓
✓	5 11 22 11 22 11	11 22	11 22	✓
✓	10 1 2 3 4 5 1 2 3 4 5	1 2 3 4 5	1 2 3 4 5	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

Check if a set is a subset of another set.

Example:

Sample Input1:

mango apple

mango orange

mango

output1:

yes

set3 is subset of set1 and set2

```
input2:
```

```
mango orange
```

```
banana orange
```

```
grapes
```

```
output2:
```

no

For example:

Test	Input	Result
1	mango apple mango orange mango	yes set3 is subset of set1 and set2
2	mango orange banana orange grapes	No

Answer: (penalty regime: 0 %)

```
import sys

def main():
    # Read all lines from standard input
    lines = sys.stdin.read().splitlines()
    if len(lines) < 3:
        return

    # Parse the three sets from space-separated input lines
    set1 = set(lines[0].strip().split())
    set2 = set(lines[1].strip().split())
    set3 = set(lines[2].strip().split())

    # Check if set3 is a subset of both set1 and set2
    if set3.issubset(set1) and set3.issubset(set2):
        print("yes")
        print("set3 is subset of set1 and set2")
    else:
        print("No")

if __name__ == '__main__':
    main()
```

	Test	Input	Expected	Got	
✓	1	mango apple mango orange mango	yes set3 is subset of set1 and set2	yes set3 is subset of set1 and set2	✓
✓	2	mango orange banana orange grapes	No	No	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

Given an array of integers `nums` containing `n + 1` integers where each integer is in the range `[1, n]` inclusive.There is only **one repeated number** in `nums`, return *this repeated number*. Solve the problem using set.

Example 1:

```
Input: nums = [1,3,4,2,2]
```

```
Output: 2
```

Example 2:

```
Input: nums = [3,1,3,4,2]
```

```
Output: 3
```

For example:

Input	Result
1 3 4 4 2	4

Answer: (penalty regime: 0 %)

```
import sys

def findDuplicate(nums):
    seen = set()
    for num in nums:
        if num in seen:
            return num
        seen.add(num)

def main():
    # Read all inputs from standard input
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    # Convert all tokens to integers
    nums = [int(x) for x in input_data]

    # Find and print the repeated number
    ans = findDuplicate(nums)
    if ans is not None:
        print(ans)

if __name__ == '__main__':
    main()
```

	Input	Expected	Got	
✓	1 3 4 4 2	4	4	✓
✓	1 2 2 3 4 5 6 7	2	2	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.